

UNIT 1 · WEEK 1

Introduction to operating systems

📅 Week 1 · dates to be confirmed </> Lab TD0 (solved + AI challenge) ~ 4 animations

MANUAL M1 · THEORY OF THE UNIT

Introduction to operating systems

Core reading for unit 1 (≈ 1 h). The figures of the course machine come from running the TD0 commands (Ubuntu 26.04, Linux 7.0).

CONTENTS

- | | | | |
|----|------------------------------|----|---|
| §1 | What is an operating system? | §4 | The boot process |
| §2 | Layers and modes | §5 | Where they are today and how they got there |
| §3 | Kernel architectures | | |

§1 What is an operating system?

An operating system is the software that sits between programs and the hardware. It has two missions:

- **Managing resources.** It shares CPU time among processes, allocates and protects memory, orders access to disks, network and screen, and manages energy. It resolves the conflicts when several programs want the same thing at once.
- **Offering a machine that is easier to use.** Instead of disk sectors, interrupts and controller registers, programs see *abstractions*: processes, files, sockets, virtual memory. A program that writes a file works the same with an SSD, a network disk or a USB stick.

It also **protects**: a program cannot read another program's memory or bypass the permissions of a file. That protection depends on the hardware having different execution modes (§2).

§2 Layers and modes

From top to bottom: **applications** (bash, a browser, your programs) → **libraries** (glibc, libssl) → **system-call interface** → **kernel subsystems** (scheduler, memory, VFS, network) → **drivers** → **hardware**. The first two layers run in **user mode**; the next three, in **kernel mode**. The boundary between them is only crossed with a system call (unit 2).

Not everything a library does needs the kernel: `printf` gathers text in a buffer and `malloc` hands out memory it already asked for. Only when the hardware is needed —really writing, asking for more memory— does the library make a system call.

Practice this → animation A1.1 · System layers

§3 Kernel architectures

DESIGN	WHAT RUNS IN KERNEL MODE	FOR	AGAINST	EXAMPLES
Monolithic	Everything: scheduler, memory, file systems, network, drivers	Fast: one call, one crossing in and one out	A bug in a driver can bring the system down	Linux (with loadable modules), the BSDs
Microkernel	The minimum: inter-process communication (IPC), scheduling, memory	Isolation: a failing driver is restarted	More crossings and messages per operation	seL4 (formally verified), QNX, MINIX 3
Hybrid	Most services; some subsystems in user mode	A compromise between the two	Inherits part of both problems	Windows NT, macOS (XNU)

Linux is monolithic but modular: the course machine has **258 loaded modules** (`lsmod`), drivers that are added and removed without rebooting. A fourth idea, the *unikernel*, compiles the application and the kernel into a single specialised image, useful in the cloud.

Practice this → **animation A1.2 · Kernel architectures**

§4 The boot process

1. **UEFI firmware.** It tests the hardware, initialises memory and buses and looks in its non-volatile memory (NVRAM) for what to boot.
2. **Boot loader** (GRUB or systemd-boot). It loads into memory the kernel (`vmlinuz`) and an initial file system (`initramfs`) with the drivers needed to mount the disk.
3. **Kernel.** It decompresses, sets up virtual memory and the processors, detects devices, mounts the root and launches the **first process, PID 1**.
4. **User space.** `systemd` (PID 1) starts the services in parallel until the login. Every process descends from it.

On the course machine, `systemd-analyze` measured: firmware 9.1 s, loader 4.9 s, **kernel 2.6 s** and user space 14.5 s (total 30.9 s). The kernel is the shortest stage.

Real output

```
$ uname -srnm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod | tail -n +2 |
wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
237
28
Startup finished in 8.661s (firmware) + 3.568s (loader) + 3.253s (kernel) + 16.571s
(userspace) = 32.053s
graphical.target reached after 16.569s in userspace.
```

Practice this → **animation A1.3 · The boot process, and TD0 · 5**

§5 Where they are today and how they got there

There are operating systems in supercomputers (since November 2017 the TOP500 list has been entirely dominated by Linux; the June 2026 No. 1, LineShine, runs Kylin OS, a Linux-based distribution), in cloud servers, in phones (Android uses the Linux kernel; iOS, the XNU kernel), in cars and medical devices (often with real-time microkernels such as QNX) and in millions of embedded devices.

ERA	KEY IDEA
1950s	Batch processing: one job at a time, no interaction.
1960s	Multiprogramming and time sharing (CTSS, Multics): several users at once, the CPU is shared.
1969–1973	Unix at Bell Labs: processes, files and pipes as simple abstractions; rewritten in C.
1980s–1990s	Personal computers; Linux (1991) and Windows NT (1993).
2007–2010s	Smartphones, virtualisation and containers in the cloud.
2025–2026	Operating systems that host AI agents with their own identity, permissions and auditing (unit 9).